Requirements and Design Document

for

Inter NIT Sports Management System

Version 3.0

Prepared by Team 1:

Srinath M P	**M251063CS**	**srinath_m251063cs@nitc.ac.in**
Harish Rana	**M250785CS**	**harish_m250785cs@nitc.ac.in**
Virendrasing Patil	**M250560CS**	**virendrasing_m250560cs@nitc.ac.in**
Achyut Prabhakar	**M250617CS**	**achyut_m250617cs@nitc.ac.in**

Project Manager: **Anu Mary Chacko**

Project Client: **NIT Calicut**

Course: **CS6103E - Software Systems lab**

Date: **28 September 2025**

Glossary

Definitions of all the terms necessary to properly interpret the design document, referencing the SRS (Section 1.4)

Term	Definition
INSMS	Inter-NIT Sports Management System – The web-based system designed to manage registrations, teams, events, and results for the Inter-NIT tournament.
NIT	National Institute of Technology – Educational institutions participating in the sports tournament.
Admin	An authorized administrator of the system. It can refer to Common Admin (global) or NIT Admin (institute-specific).
Common Admin	The super-administrator responsible for approving NIT registrations, scheduling fixtures, and validating system-wide operations.
NIT Admin	An administrator from a specific NIT.
Coach	Faculty or staff member responsible for registering teams, managing rosters, and overseeing player participation.
Student/User	General participants or spectators who access results, schedules, announcements, and leaderboards (read-only role).
CRUD	Create, Read, Update, Delete – Basic operations for data management.
JWT	JSON Web Token – A secure token standard used for authentication and role-based access control.
MERN	MongoDB, Express.js, React.js, Node.js – The core technology stack used to build the system.
API	Application Programming Interface – Enables communication between frontend and backend services.
UI	User Interface – The visual elements and layouts that allow users to interact with the system.
UX	User Experience – The overall ease, efficiency, and satisfaction with which users interact with the application.
HTTPS	HyperText Transfer Protocol Secure – Protocol used for encrypted communication between client and server.
ODM	Object Data Modeling – A method (via Mongoose) to define schemas and interact with MongoDB.
DBMS	Database Management System – Software (MongoDB) used to store, retrieve, and manage data.
CDN	Content Delivery Network – Distributed servers (used in Vercel) to speed up content delivery globally.
GDPR	General Data Protection Regulation – EU law for data protection and privacy compliance.

Table of contents

Glossary 2

Table of contents 3

MERN Application Design Document: Inter NIT Sports Management System 4

1. Introduction 4

2. Functional Requirements 5

3. UI Design 5

4. System Architecture 10

4.1. High-Level Diagram 11

4.2. Technology Stack 12

4.3. Database Schema (MongoDB Collections) 14

Database Schema (MongoDB Collections) **Error! Bookmark not defined.**

Collection: Users 14

Collection: NITs 15

Collection: Teams 15

Collection: Events 15

Collection: Matches 16

Collection: Notifications 16

Collection: AuditLog 16

MERN Application Design Document: Inter NIT Sports Management System

1. Introduction

Goal: The primary goal of this project is to design and develop the Inter-NIT Sports Management System (INSMS), a centralized, web-based platform to efficiently manage the annual Inter-NIT tournament. The system will replace traditional manual processes with a streamlined digital solution.

- **Project Vision:** To provide a single, reliable, and user-friendly digital platform for organizing all aspects of the Inter-NIT sports tournament, from registration to result publication.

- **Problem Statement:** The management of the Inter-NIT tournament currently relies on manual processes like paper-based registrations, email communication for scheduling, and offline methods for recording results. This approach is inefficient, prone to errors, and lacks a centralized source of information for participants and organizers.

- **Target Audience:** The primary users of this application are:
    - **Common Admins:** The central organizers who approve NIT participation and oversee the tournament.
    - **NIT Admins:** Administrators from each participating NIT responsible for managing their institute's events, teams, and information publication.
    - **Coaches:** Users who register and manage their respective teams and players.
    - **General Users (Students/Spectators):** Anyone who wishes to view schedules, results, announcements, and leaderboards without needing to log in.

2. Functional Requirements

Key Features & Scope: A high-level list of the main features that will be built.

- **Feature A: NIT Registration Management** – Approve or reject NIT registrations. *(F1)*
- **Feature B: Team Registration** – Coaches register teams for sports. *(F2)*
- **Feature C: Event Scheduling** – NIT Admins create events with venue. *(F3)*
- **Feature D: Accommodation & Mess Management** – Hosting NITs publish stay and food details. *(F4)*
- **Feature E: Match Results Recording** – NIT Admins record winners, scores, and remarks. *(F5)*
- **Feature F: Leaderboards & Reports** – Generate leaderboards and performance reports. *(F6)*
- **Feature G: Notification Services** – Email/SMS updates to users. *(F7)*
- **Feature H: Role-Based Access Control** – Different permissions for Common Admin, NIT Admin, Coach, Viewer. *(F8)*
- **Feature I: Secure Data Storage** – Store team and match data in MongoDB. *(F9)*
- **Feature J: Dashboards** – Display schedules, results, and announcements. *(F10)*
- **Feature K: Schedule Validation** – Common Admin validates NIT-proposed schedules. *(F11)*
- **Feature L: Audit Trail** – Maintain logs of all admin actions. *(F12)*
- **Feature M: Fixture Management** – Common Admin sets fixture pairings manually. *(F13)*
- **Feature N: Report Export** – Export reports in PDF/Excel. *(F14)*
- **Feature O: Input Data Validation** – Ensure error-free and consistent data entry. *(F15)*

Note:

3. UI Design

A mock-up of the user interface of core features that are yet to be implemented.

3.1 Login Page:

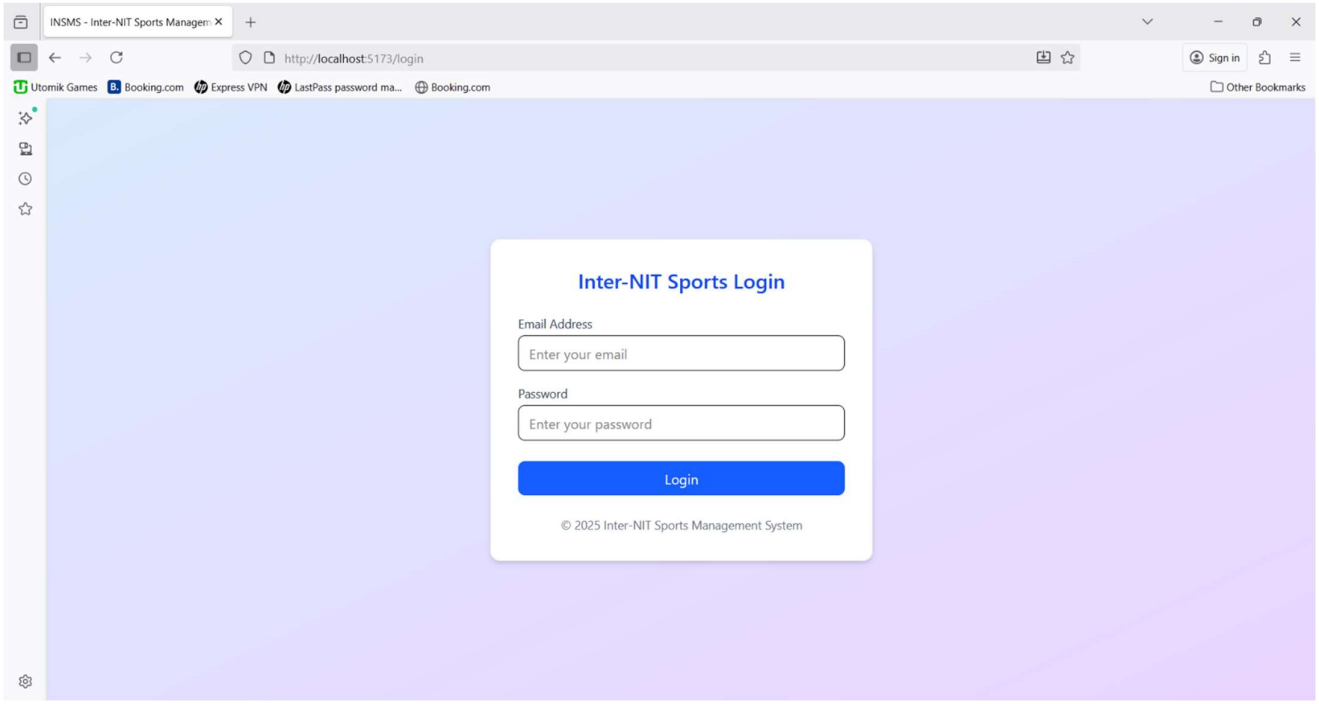


Figure 1: Login page

3.2 Common-Admin dashboard:

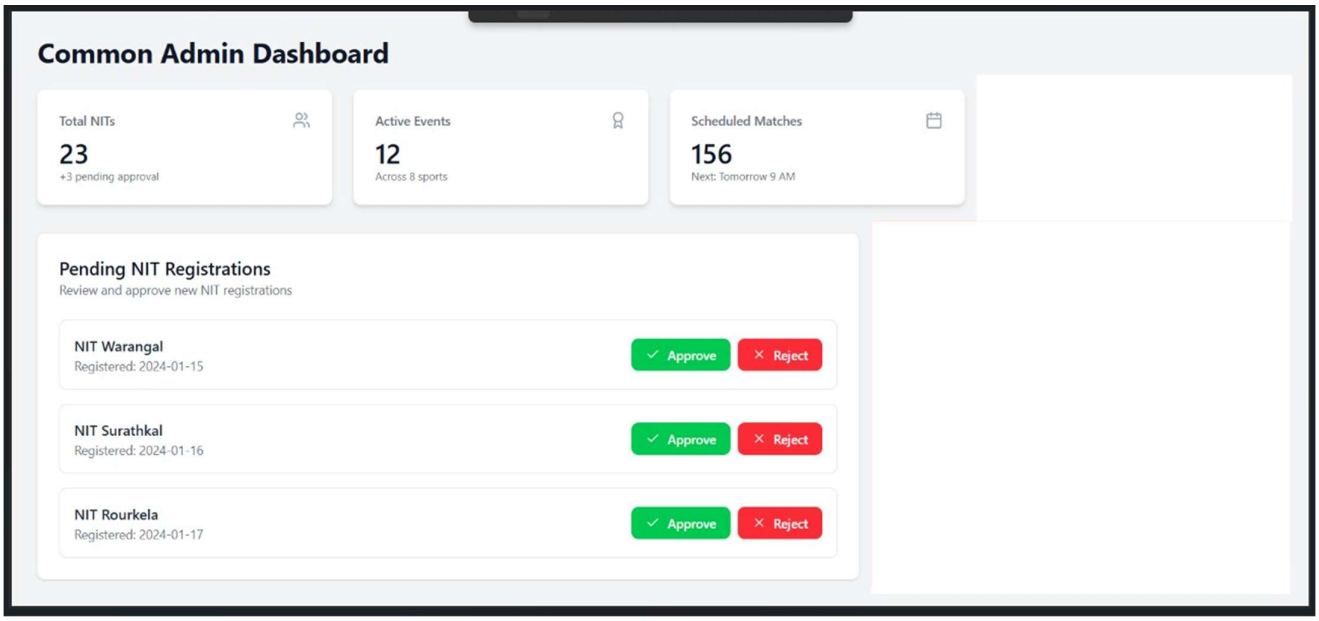


Figure 2: Common admin dashboard

3.3 NIT Admin dashboard

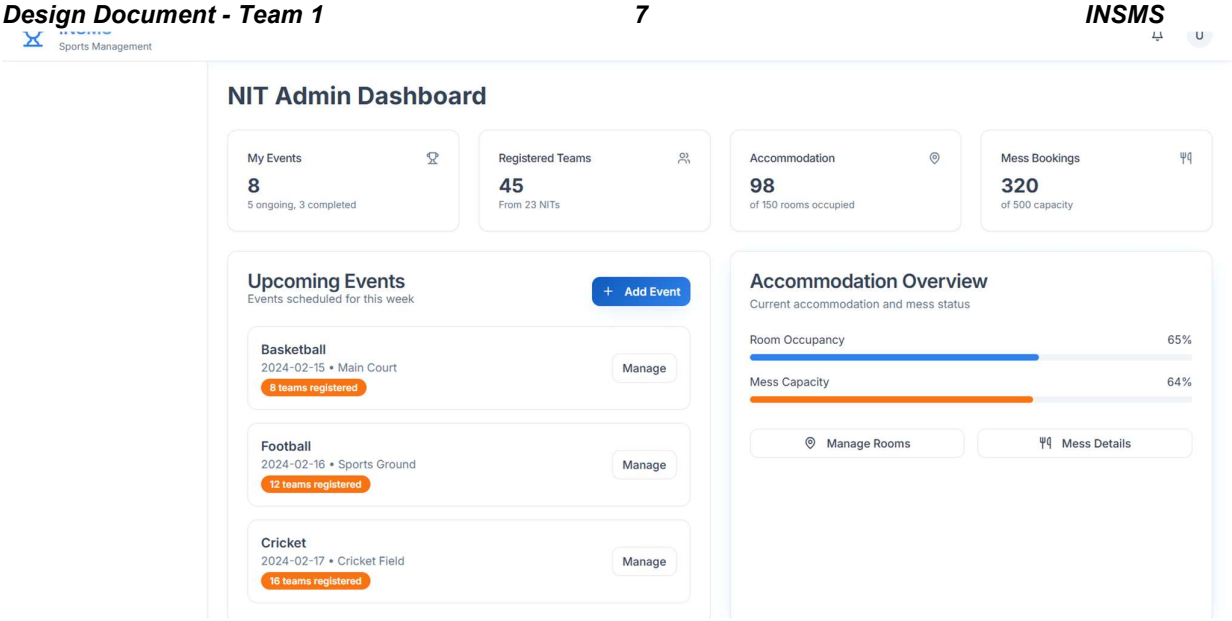


Figure 3: NIT admin dashboard

3.4 Coach Dashboard

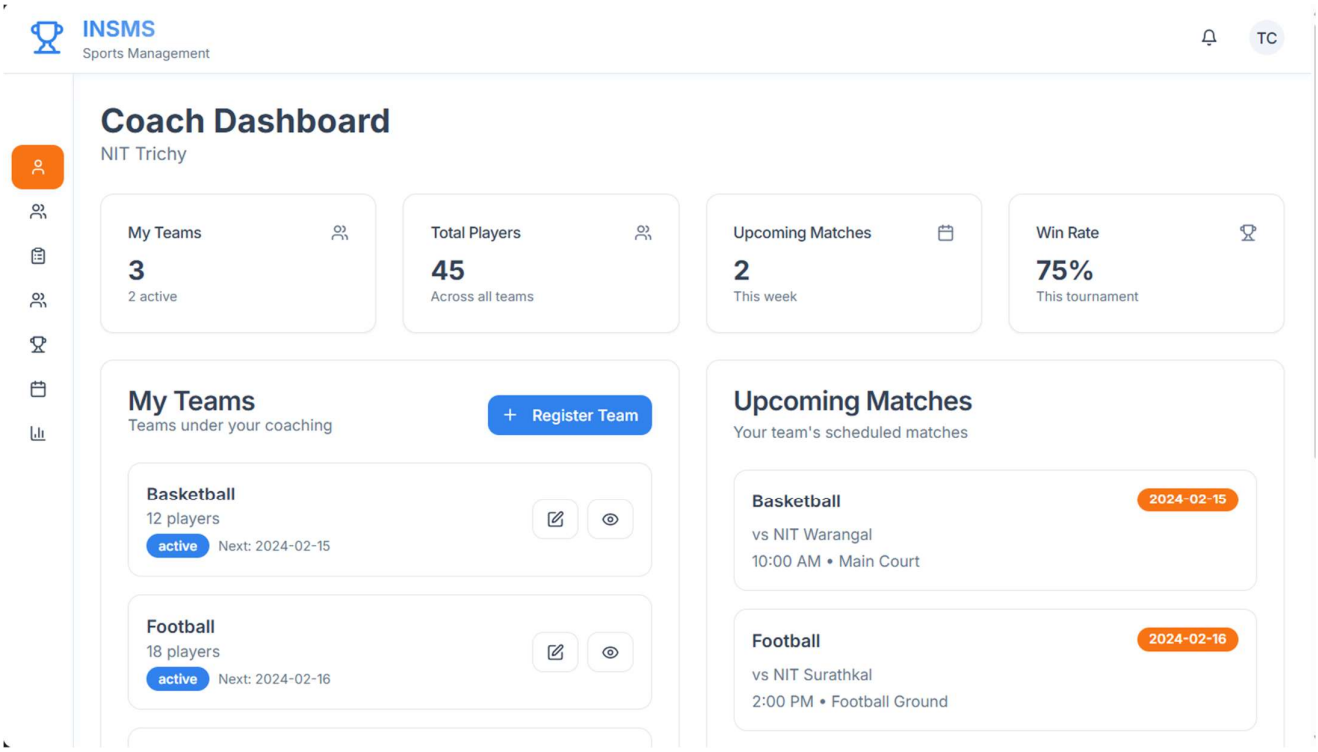


Figure 4: Coach dashboard -1

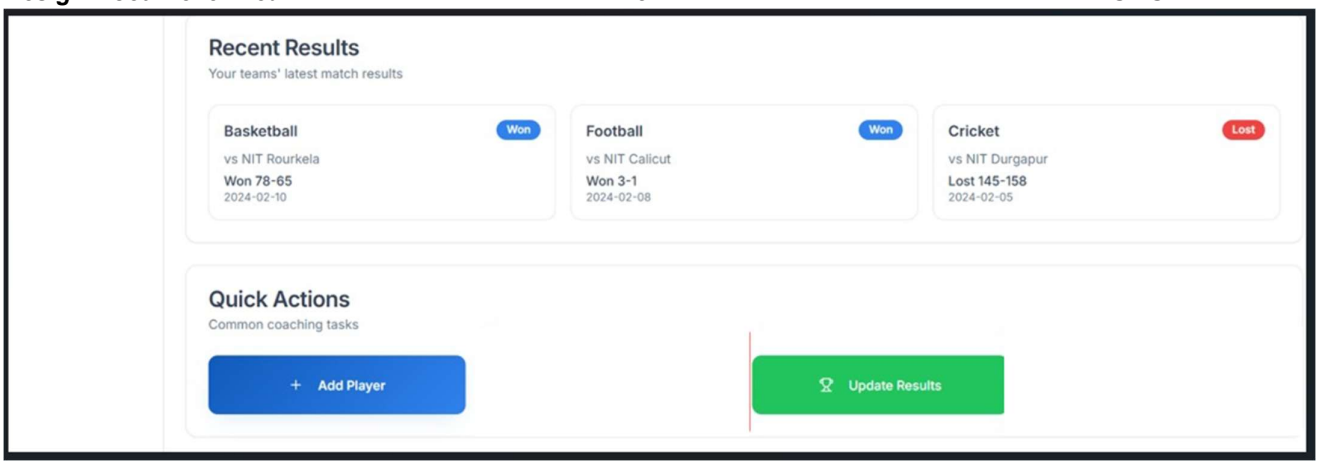


Figure 5: Coach dashboard -2

3.5 Team management

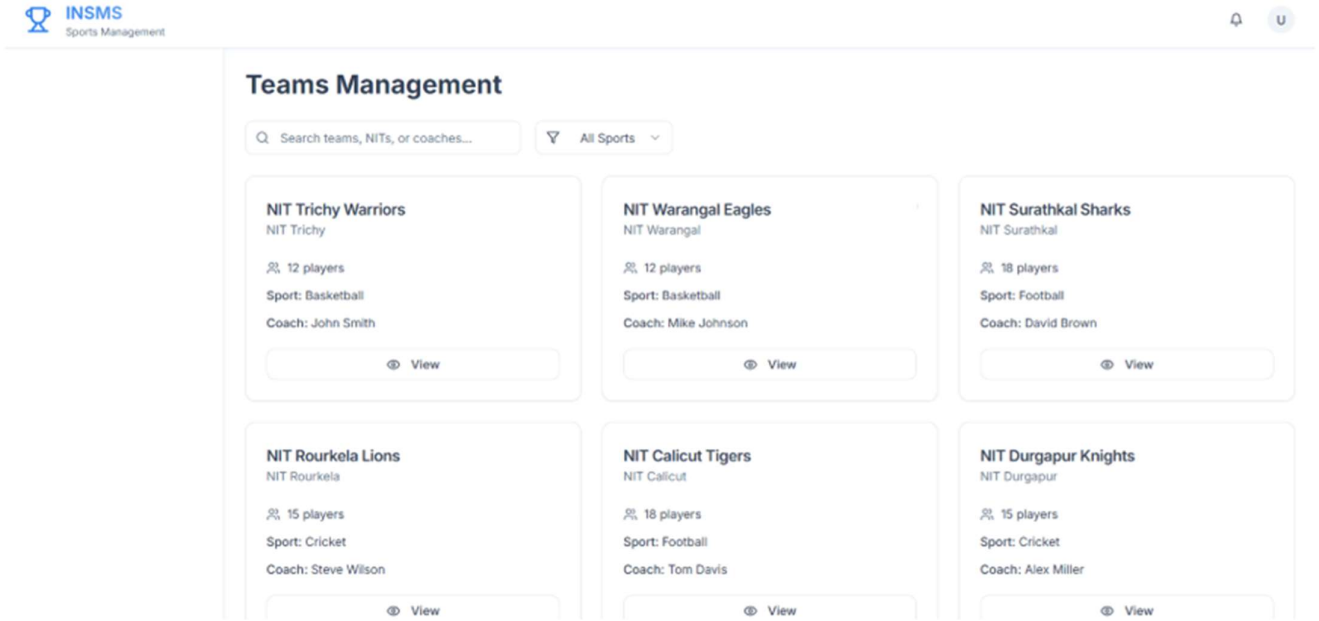


Figure 6:Team management -1

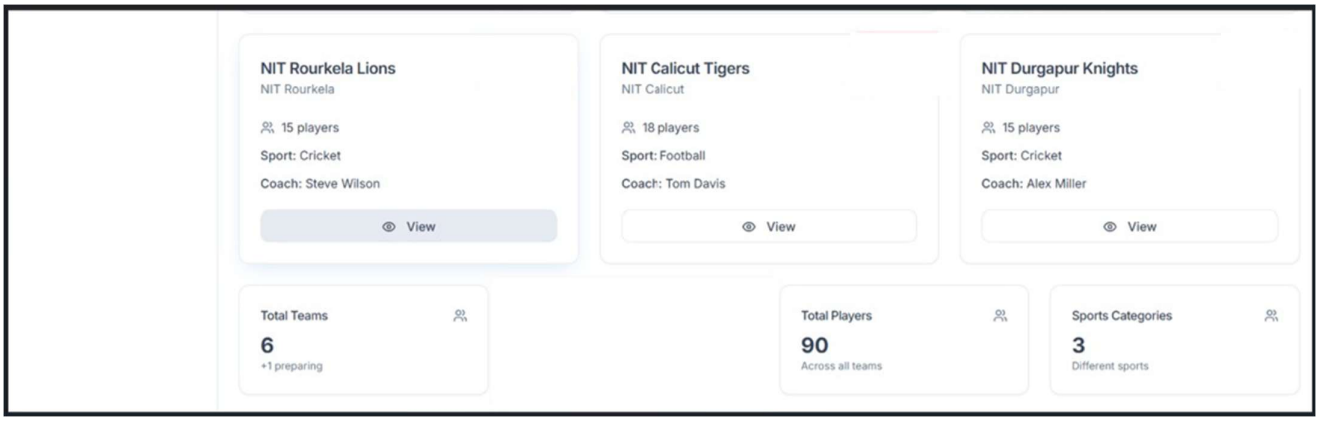


Figure 7: Team management -2

3.6 Events Management

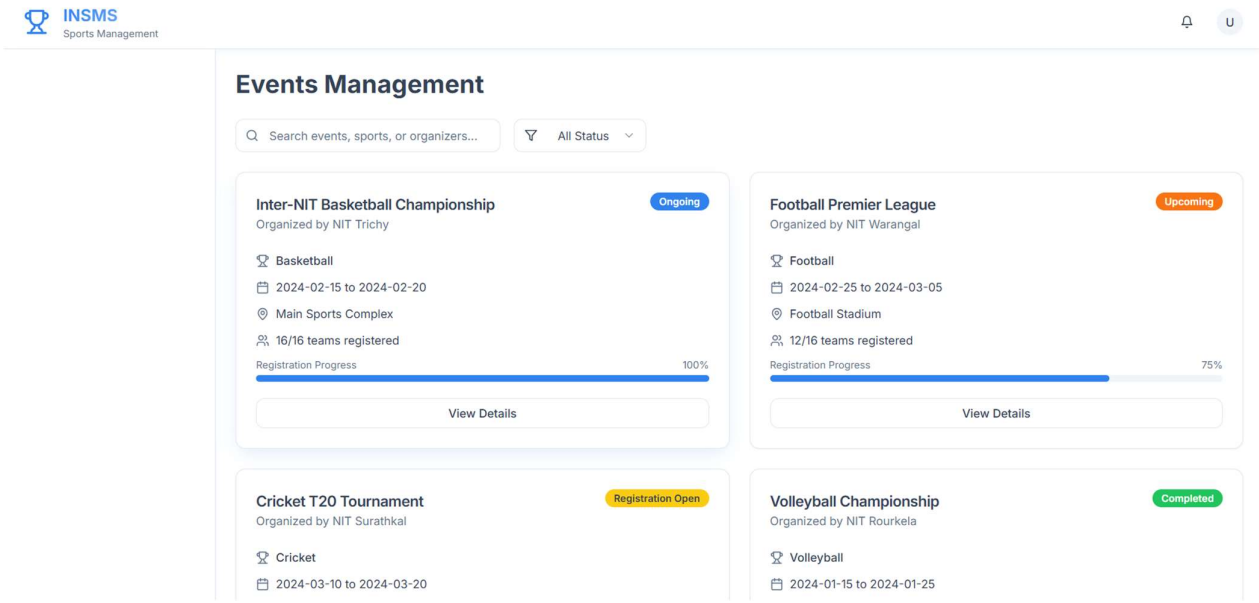


Figure 8: Event management

3.7 Matches and fixtures

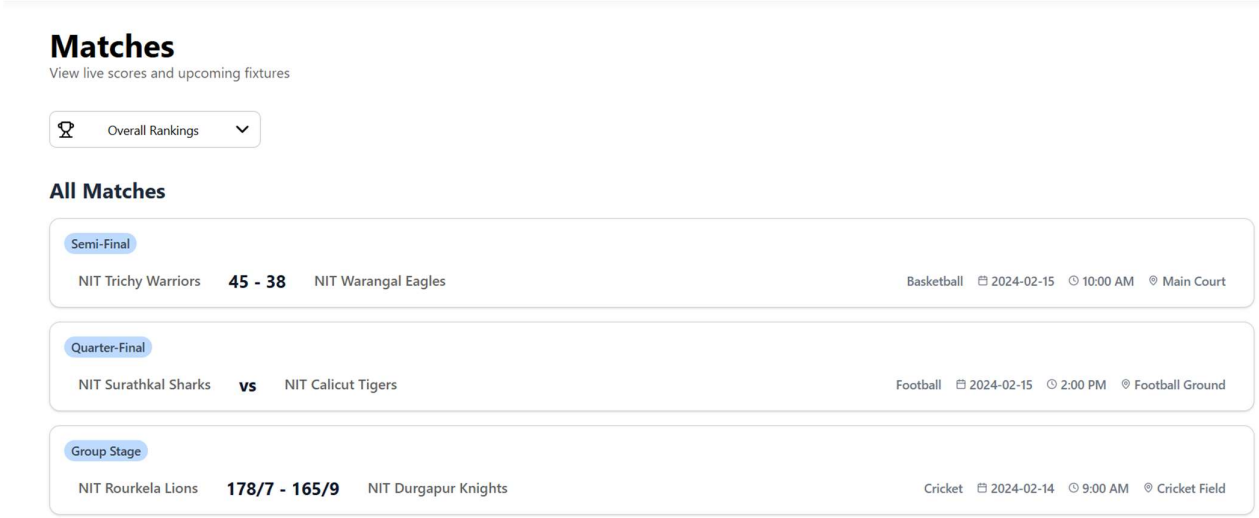


Figure 9: Matches and fixtures -1

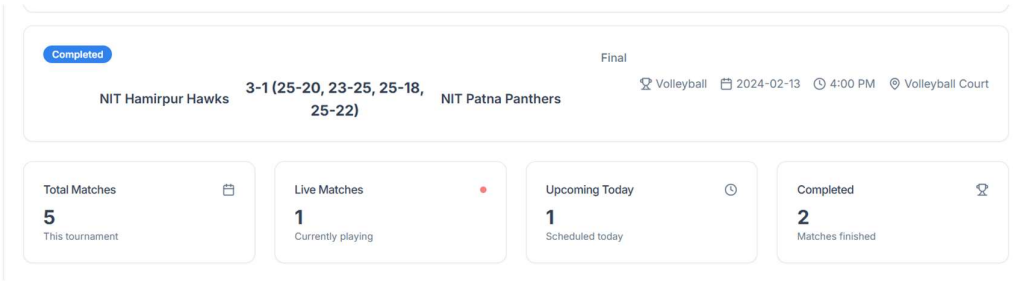


Figure 10: Matches and fixtures -2

3.8 Leaderboard and ranking

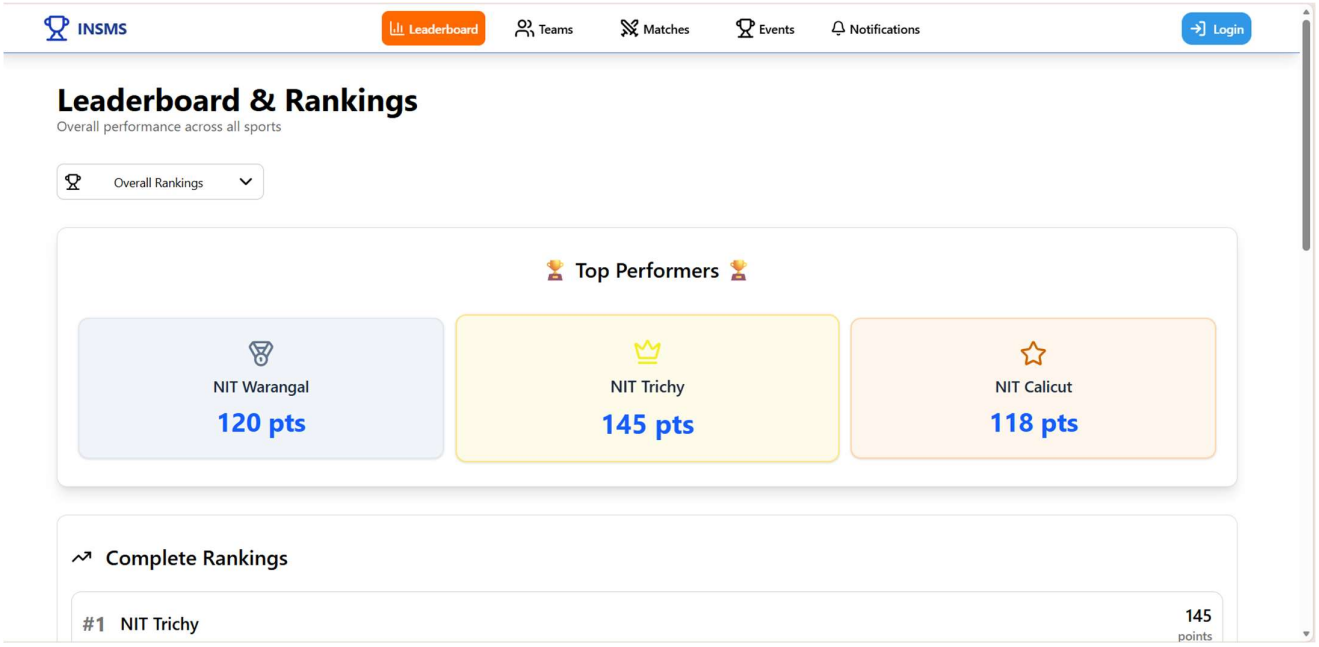


Figure 11: Leaderboard and ranking

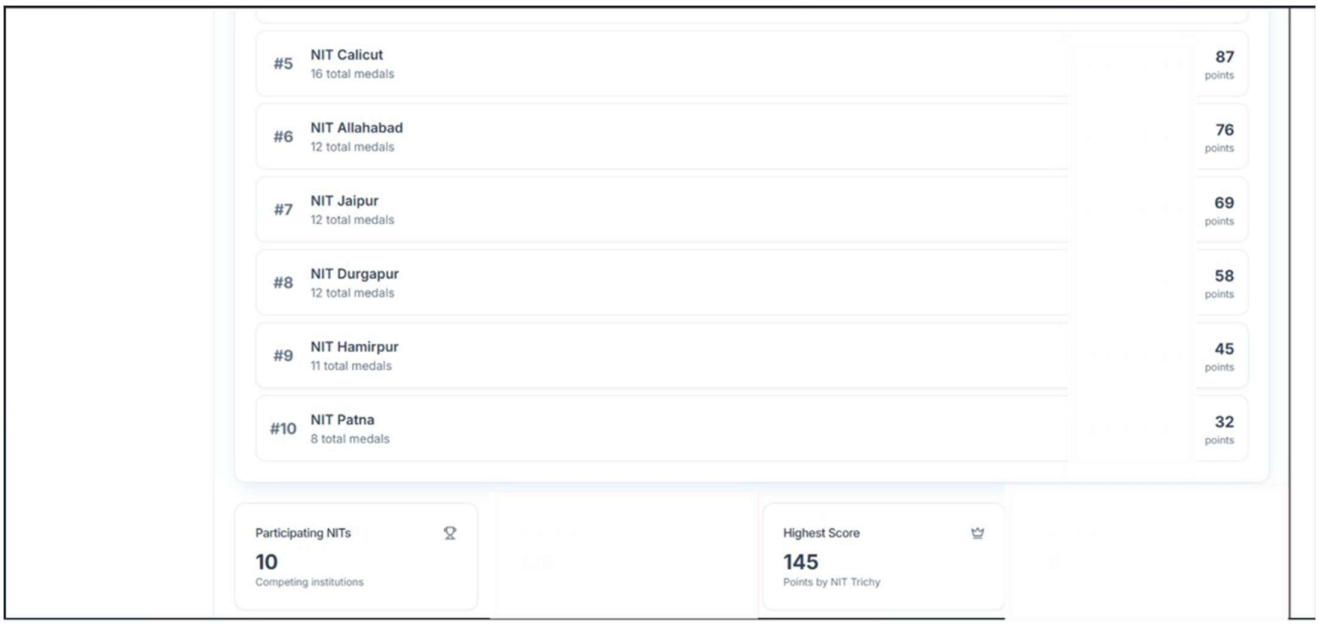
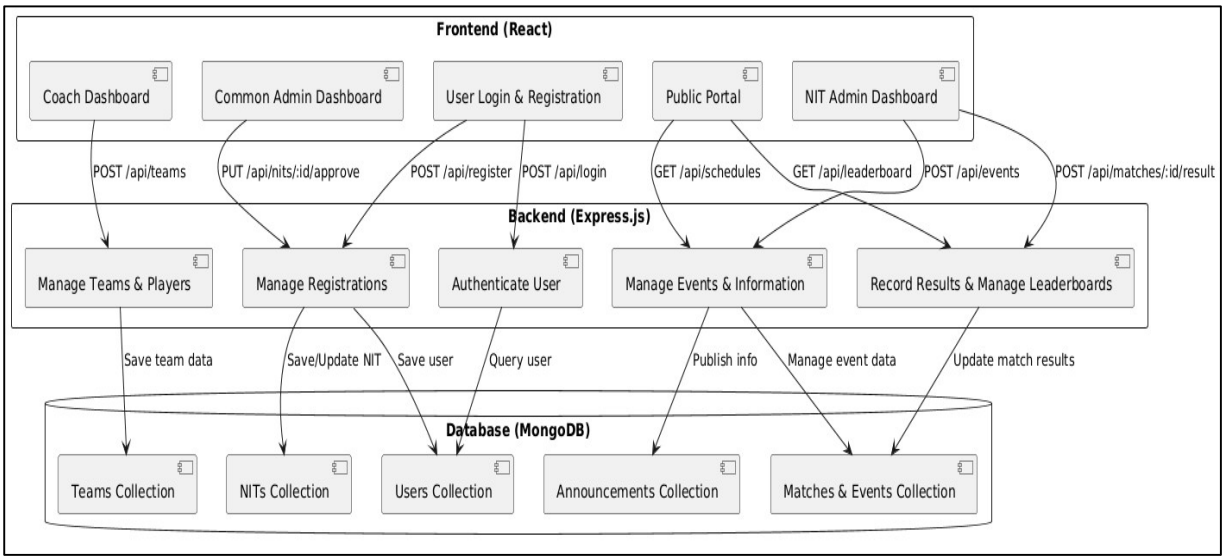


Figure 12: Leaderboard and ranking -2

4. System Architecture

Goal: Outline the overall structure, technology choices, and how the different parts of the MERN stack interact.



4.1. High-Level Diagram

A visual representation of the application's architecture.

Client (React): A single-page application (SPA) running in the user's browser. It provides the user interface for managing the core entities of the system, such as **Teams, Events, and Matches**, and communicates with the backend via a REST API.

Server (Node.js/Express): A RESTful API server that handles all business logic. This includes managing **Users** and **NITs**, processing **Team** registrations, scheduling **Events** and **Matches**, and publishing **Announcements**. It interacts with the database to perform these operations and manages user authentication.

Database (MongoDB): A NoSQL database used to store all application data. The data is organized into collections corresponding to the main classes, such as **Users, NITs, Teams, Events, Matches, and Announcements**, and is hosted on a service like MongoDB Atlas.

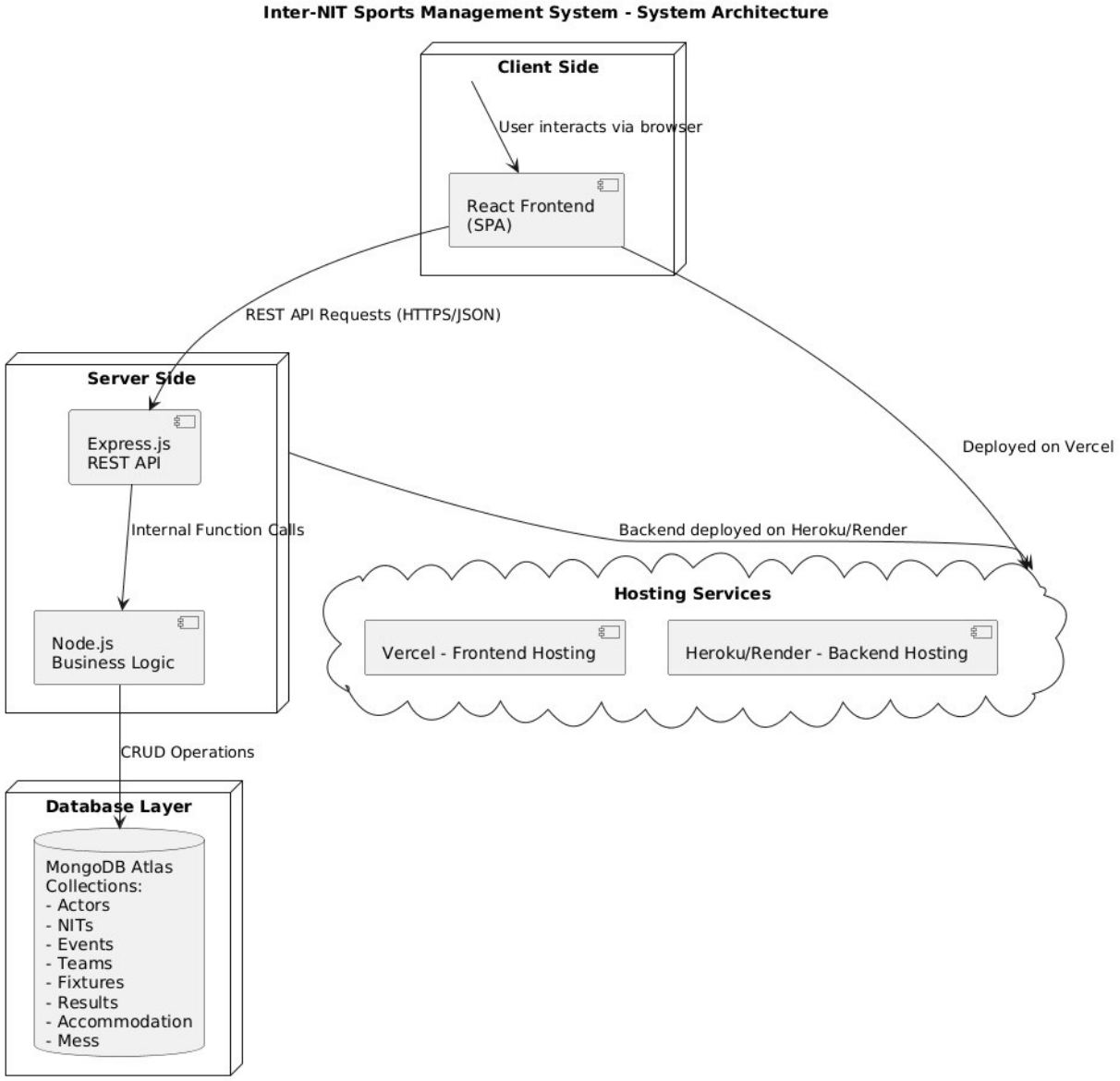


Figure 13: High level system architecture

4.2. Technology Stack

Frontend

- **React 18.x** – Provides a modular, component-based architecture for building dynamic, interactive user interfaces.
- **React Router 6.x** – Enables seamless navigation and routing between different pages and dashboards for various user roles.
- **Axios** – Used for efficient API communication between frontend and backend services.
- **Tailwind CSS** – A utility-first CSS framework for responsive, mobile-friendly, and consistent styling across all views.

Backend

- **Node.js 20.x** – A high-performance, event-driven JavaScript runtime environment for handling concurrent requests.

- **Express 4.x** – A lightweight web framework used to build RESTful APIs, manage routes, and handle middleware.
- **Mongoose 8.x** – An ODM (Object Data Modelling) library for MongoDB, ensuring schema validation and smooth database interaction.

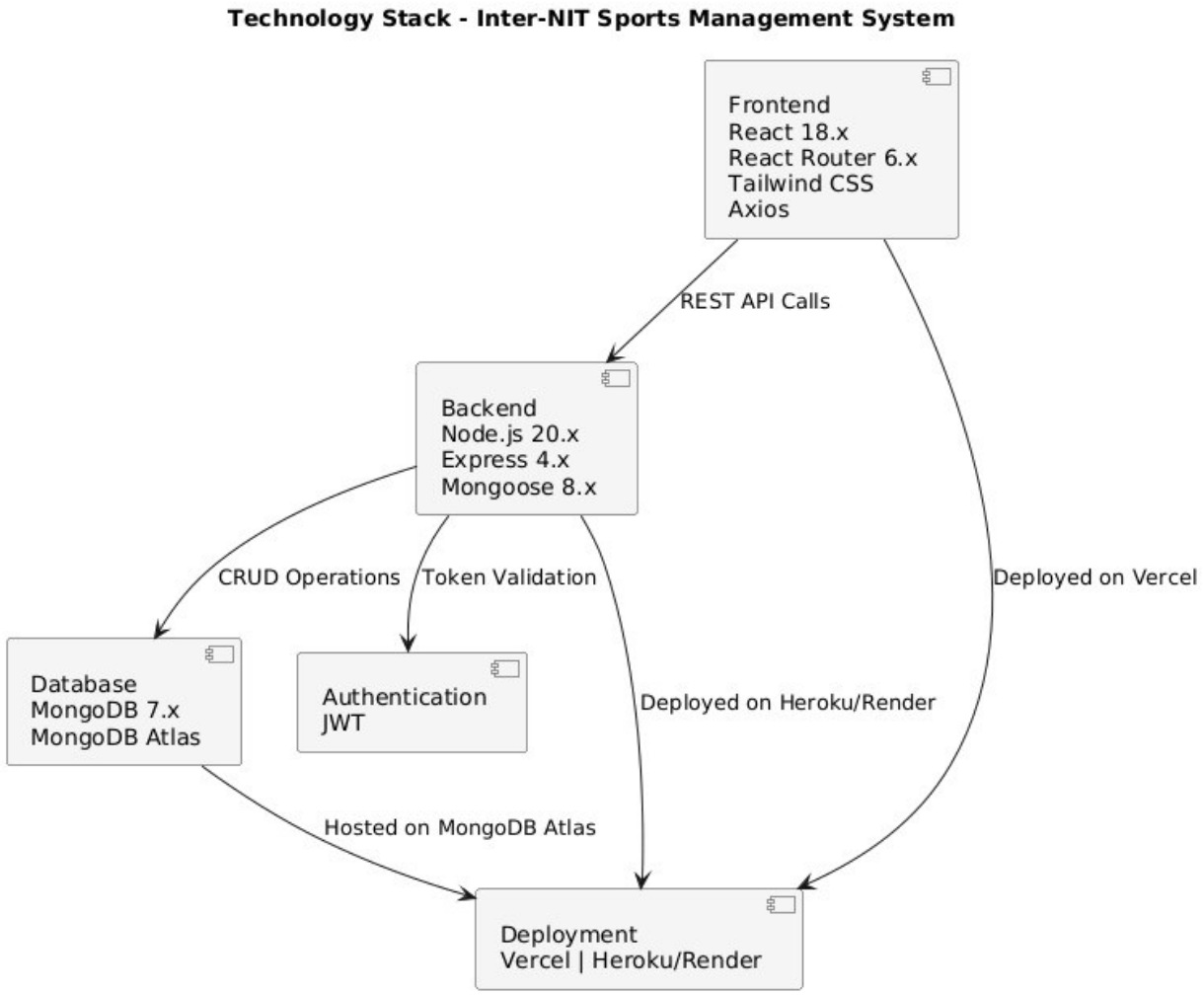


Figure 14: Technology stack

Database

- **MongoDB 7.x** – A NoSQL database that provides scalability and flexibility for handling tournament data, including users, teams, matches, and results.
- **MongoDB Atlas** – A fully managed cloud-hosted solution ensuring high availability, automated backups, and global scalability.

Authentication & Security

- **JSON Web Tokens (JWT)** – Provides secure role-based authentication and authorization for all system users (Common Admin, NIT Admin, Coach, Student).
- **HTTPS & Password Hashing** – Ensures encrypted communication and secure password storage.

Deployment

- **Vercel (Frontend)** – Offers fast, serverless deployment for React applications with global CDN for optimized delivery.
- **Heroku (Backend)** – Provides a scalable cloud platform to host and manage Node.js/Express APIs.
- **MongoDB Atlas (Database)** – Cloud-based hosting of the database, ensuring reliability and performance under tournament load.

4.3. Database Schema (MongoDB Collections)

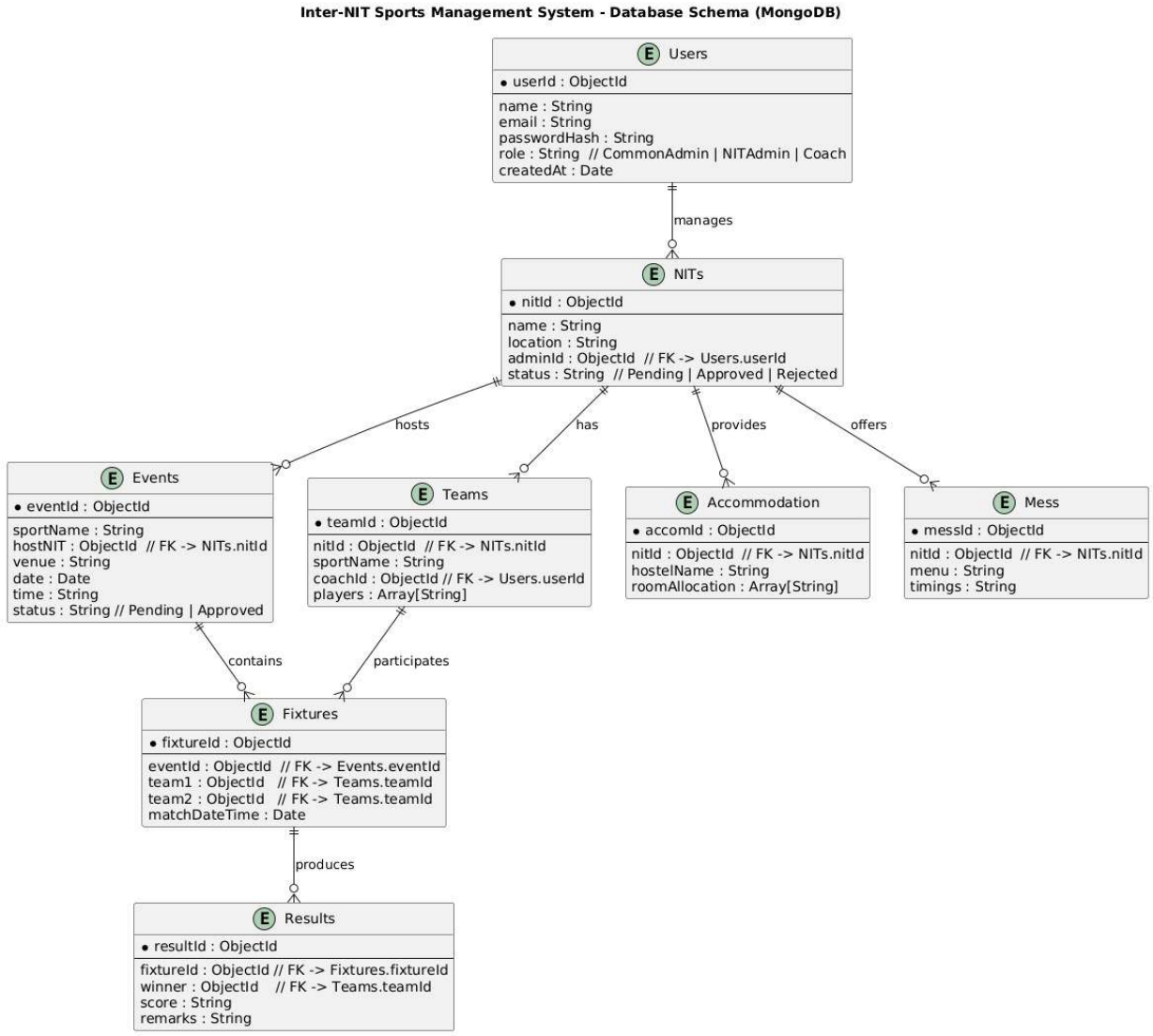


Figure 15:Database Schema

4.3.1. Collection: Users

Field Name	Data Type	Description
_id	ObjectID	Unique User ID
name	String	Full name of the user
email	String	User email (unique, used for login)
passwordHash	String	Hashed password (bcrypt/argon2)
role	String	User role: CommonAdmin, NITAdmin, Coach, Student

nit_id	ObjectID	FK to NIT collection (for Admins/Coaches/Students)
isActive	Boolean	Indicates activeness

4.3.2. Collection: NITs

Field Name	**Data Type**	**Description**
_id	ObjectID	Unique NIT ID
name	String	Name of the NIT
location	String	City/State of NIT
status	String	Approval status (Pending, Approved, Rejected)
created_at	Date	Registration date
points	Number	Points scored by teams of NITs

4.3.3. Collection: Teams

Field Name	**Data Type**	**Description**
_id	ObjectID	Unique Team ID
name	String	Team name
sport	String	Sport name (e.g., Cricket, Football, Basketball)
coach_id	ObjectID	FK to Users (Coach)
nit_id	ObjectID	FK to NITs
players	Array	List of players (player_id references Users)
players[].name	String	Player Name
players[].jerseyNo	Number	Jersey Number
players[].position	String	Position

4.3.4. Collection: Events

Field Name	**Data Type**	**Description**
_id	ObjectID	Unique Event ID
name	String	Event name (e.g., Cricket Tournament)
sport	String	Type of sport
venue	String	Venue of event
datetime	DateTime	Scheduled date & time
created_by	ObjectID	FK to NIT Admin (User ID)
status	String	Scheduled / Completed / Cancelled
tournamentYear	Number	Tournament Year

stage	String	Stage

4.3.5. Collection: Matches

Field Name	**Data Type**	**Description**
_id	ObjectID	Unique Match ID
event_id	ObjectID	FK to Event
teamA_id	ObjectID	FK to Team A
teamB_id	ObjectID	FK to Team B
scoreA	String	Final score of team A
scoreB	String	Final score of team B
winner_id	ObjectID	FK to the winning team
status	String	Scheduled, Ongoing, Completed
remarks	String	Remarks of the match
matchDateTime	Datetime	Date & time of match

4.3.6. Collection: Notifications

Field Name	**Data Type**	**Description**
_id	ObjectID	Unique Notification ID
user_id	ObjectID	FK to User (if targeted)
message	String	Notification message
type	String	Announcement, Result
created_by	ObjectID	FK to recipent
recipientRole	String	Role of Recipent
isRead	Boolean	Indicates if read or not

4.3.7. Collection: AuditLog

Field Name	**Data Type**	**Description**
_id	ObjectID	Unique Log ID
action	String	Action performed (Create, Update, Delete, Approve)
user_id	ObjectID	FK to User
entity	String	The type of document that was changed
entity_id	ObjectID	The specific document ID
details	String	A short description of the change

4.3.8 Collection: Accommodation

Field Name	Data Type	Description
_id	ObjectID	Unique Log ID
nit_id	ObjectID	FK to NIT
hostel_name	String	Name of hostel
room_number	Number	Room number
capacity	Number	Capacity of the room
occupied	Number	Current occupancy
team_id	ObjectID	FK to Team
check_in_date	Datetime	Check-in time
Check_out_date	Datetime	Check-out time
remarks	String	Remarks

4.3.8 Collection: Mess

Field Name	Data Type	Description
_id	ObjectID	Unique Log ID
nit_id	ObjectID	FK to NIT
mess_name	String	Name of mess
capacity_per_meal	Number	Mess capacity
capacity	Number	Capacity of the room
location	String	Current mess location
meal_schedule	ObjectID	FK to meal schema
allocated_teams	ObjectID	Teams allocated to mess
remarks	String	Remarks

5. **Control and Data Flows through Activity Diagrams**

5.1 NIT Registration → Common Admin Approval:

Corresponds to functional requirement:

F1: The system shall allow Common Admins to approve or reject NIT registrations

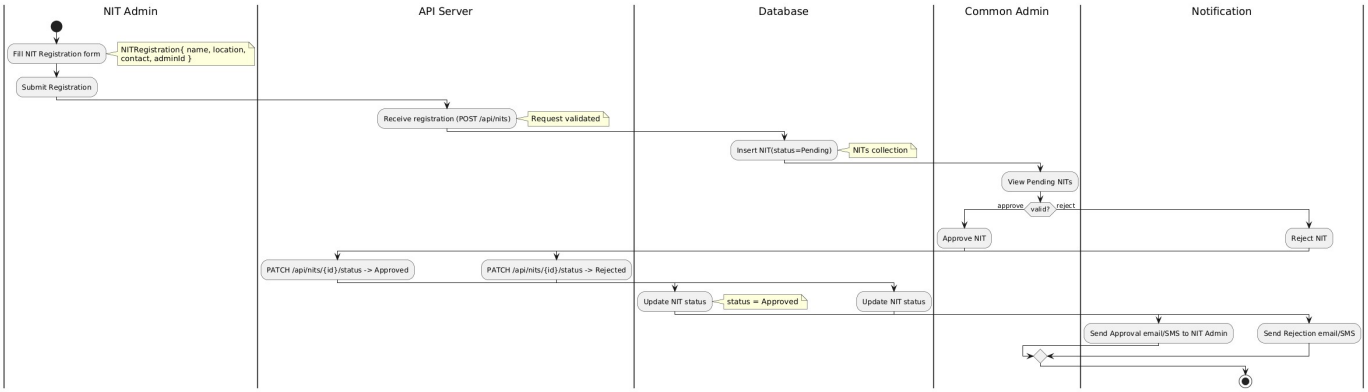


Figure 16: NIT Registration

The figure(16) diagram shows how an NIT Admin submits a registration request. A **valid registration** requires: (i) unique NIT name, (ii) valid admin email ID, and (iii) all mandatory fields filled (location, contact details). The Common Admin then reviews and either approves or rejects the request.

5.2 Event Scheduling by NIT Admin → Common Admin Validation

Corresponds to functional requirements:
F3: The system shall allow NIT Admins to create Events with venue and time
F11: The System shall allow Common Admin to schedule events requested by host-NIT Admin.

This flow illustrates how NIT Admins create events with a venue. A **valid schedule** is one without time conflicts, with an approved venue, and aligned to the tournament calendar. The Common Admin validates before publication.

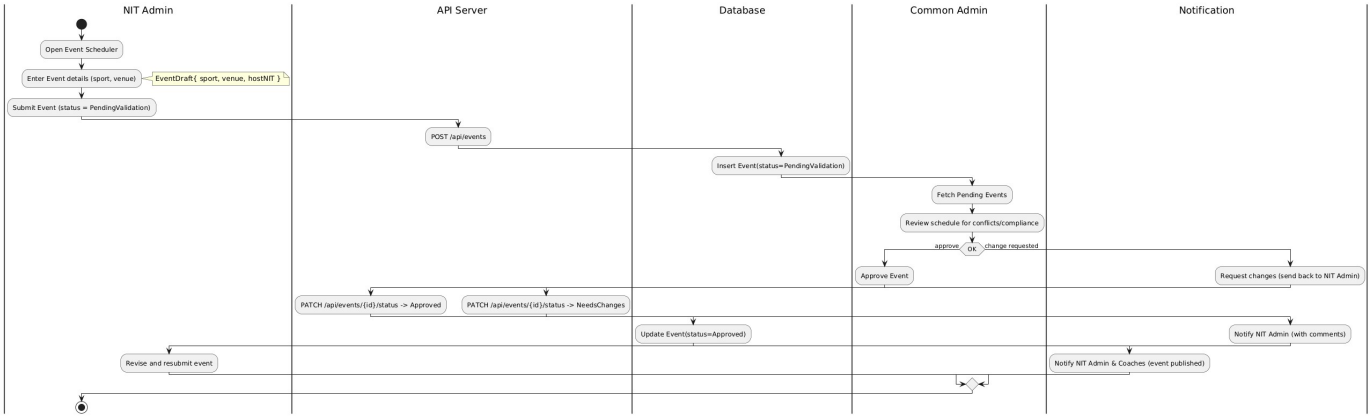


Figure 17: Event Scheduling

5.3 Coach Registers Team

Corresponds to functional requirements:
F2: The system shall allow Coaches to register teams and players.

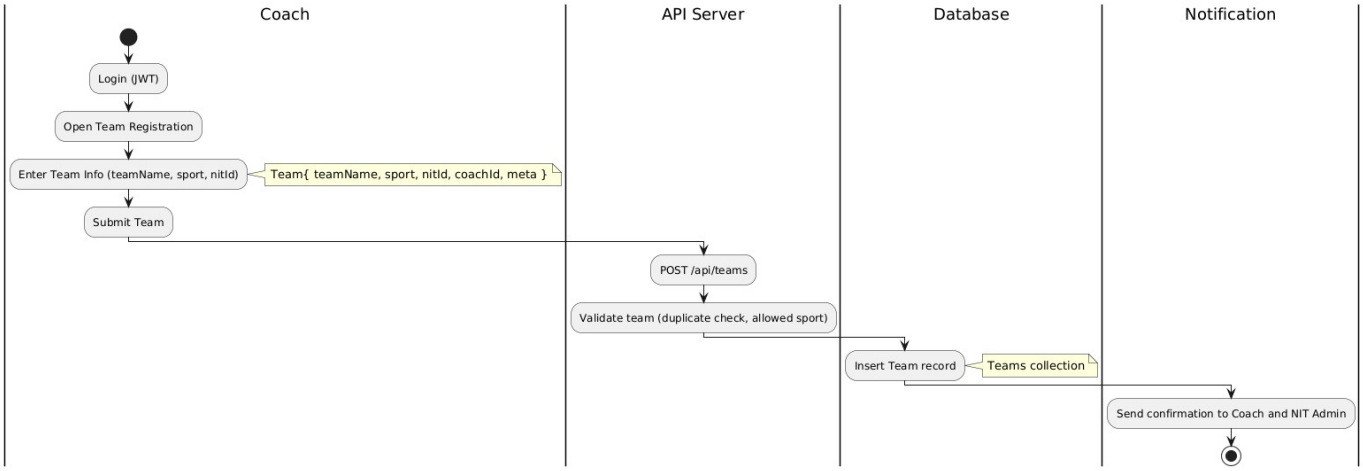


Figure 18: Coach registering team

5.4 Fixture Management → Match Result Recording & Publication

Corresponds to functional requirements:
F5: The system shall allow NIT Admins to record match results including winner, score, and remarks.
F11: The System shall allow Common Admin to schedule events requested by host-NIT Admin.
F13: The system shall allow Common Admins to manually set fixture pairings.

This diagram describes fixture setup by the Common Admin and result submission by the NIT Admin. A **valid match result** includes fixture ID, both team IDs, final score, winner, and remarks. Results remain "pending" until validated by the Common Admin.

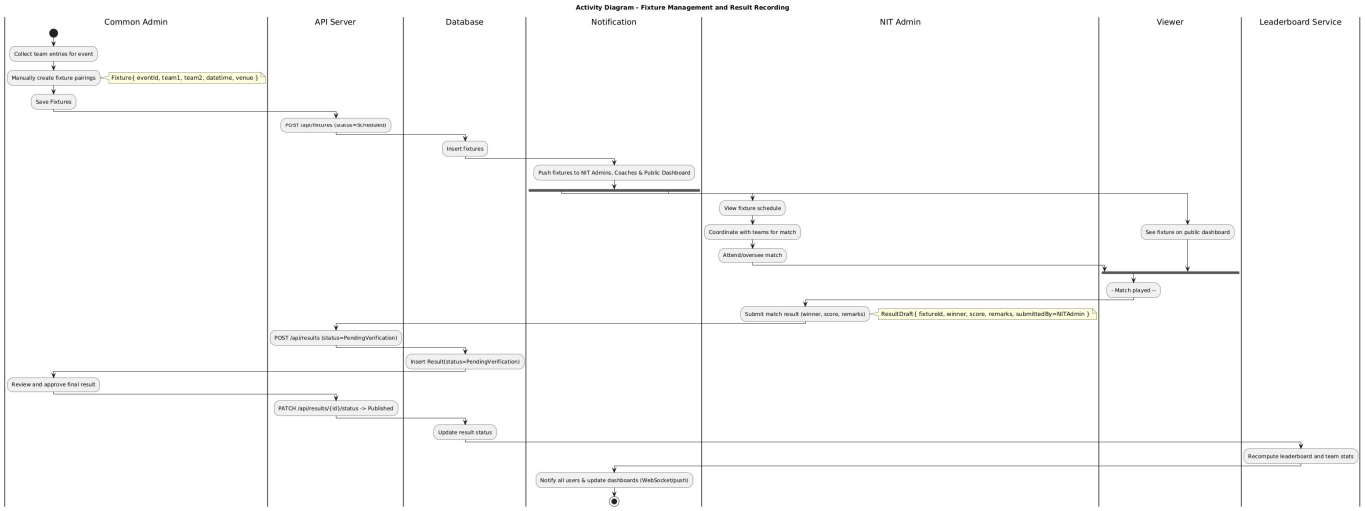


Figure 19: Fixture Management

5.5 Viewer Dashboard / Leaderboard Generation

Corresponds to functional requirement:
F6: The system shall generate leaderboards and reports for teams and players.
F10: The system shall display schedules, results, and announcements on user dashboards.
F12: The system shall maintain an audit trail of all administrative changes.
F14: The system shall provide an export option for reports in PDF or Excel format.

Leaderboard generation uses win/loss records, points, and tie-breaking rules (head-to-head, score difference). The system recomputes rankings automatically after each approved match result.

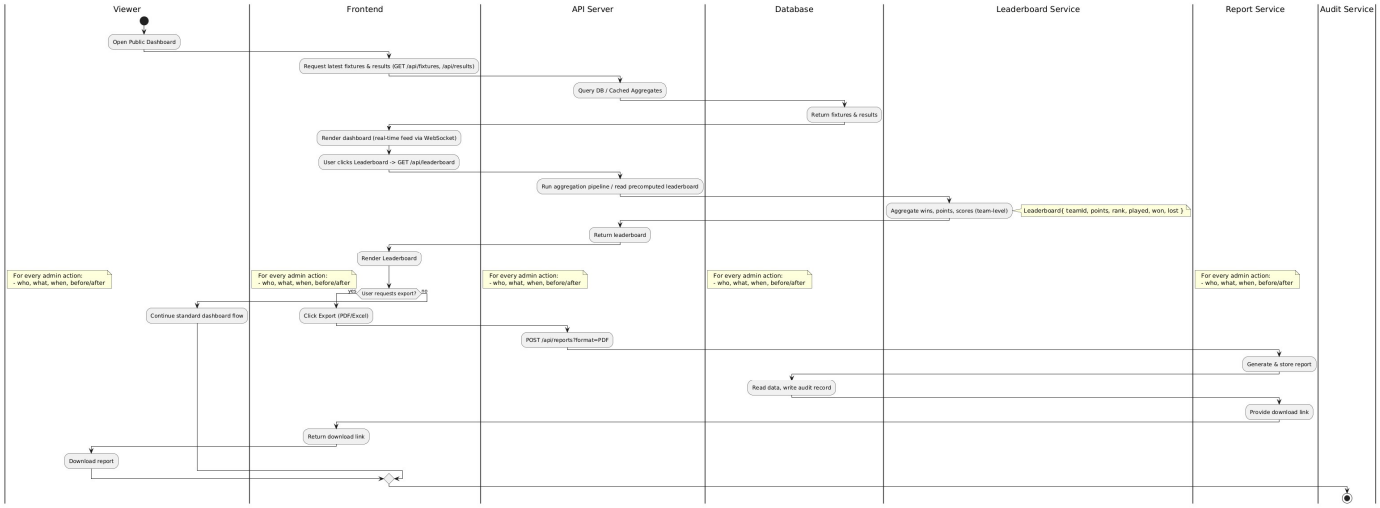


Figure 20: Viewer Dashboard / Leaderboard Generation

References

1. **Websites**:

ReactJS. (2024). React - A JavaScript library for building user interfaces. React. Retrieved from https://reactjs.org/

2. **Articles** and Online Resources:

JavaScript Info. (2023). How to use WebSocket in JavaScript. JavaScript.info. Retrieved from https://javascript.info/websocket

3. API Documentation:

Socket.io. (2024). Socket.IO Documentation. Retrieved from https://socket.io/docs/

MongoDB. (2024). MongoDB Atlas: Cloud-hosted MongoDB (Version 6.0). Retrieved from https://www.mongodb.com/cloud/atlas

4. Tools and Libraries:

Axios. (2024). Promise based HTTP client for the browser and node.js. Retrieved from https://axios-http.com/

5. Framework Documentation:

Express.js. (2024). Express - Node.js web application framework. Retrieved from https://expressjs.com/